Advanced Graphics Programming in Unreal, part 3

9 min read · Jun 24, 2025

William Mishra-Manning

This is a seven-part article series.

Part 1: Introduction
Part 2: Rendering Abstractions (RHI, RDG) and Threading
Part 3: Scenes. Views, and SceneViewExtension
Part 4: Global Shaders
Part 5: Simple Material Shaders
Part 6: Advanced Material Shaders
Part 7: Mesh-Material Shaders and Mesh Passes

Scenes and Views

The last step before we start writing our own render passes is to understand how Unreal organizes scene and view data. The major data types are listed here from outermost to innermost: Scene, Scene Renderer, View Family, and View.

We also need to go over per-view data management and *Scene-View Extensions*, which are custom hooks into the render pipeline. They are the building blocks of a custom render pass!

Theory

Scene

`FSceneInterface`, and its concrete child `FScene`, are the render-thread equivalent of a `UWorld`. They hold a list of the world's render-related components (meshes, decals, lights, etc.), with their associated proxies which are periodically updated and batched.

The concrete `FScene` also contains numerous collections and GPU resources needed to render the world. For example, it holds the directional lights data in the array `DirectionalLights`. It also owns an instance of the inner struct `FGPUScene`, which contains the GPU-side buffers.

Scene Renderer

An `FSceneRenderer` is a short-term object, allocated to render a single frame of a scene. It owns a group of view-families and views (see below). I have not directly interacted with this type, but it's there.

View Family

An `FSceneViewFamily` represents a single configured viewport (player view, Scene Capture Component, etc), rendering to either the screen or a texture. In practice it is one of two child types:

- `FSceneViewFamilyContext` for short-term uses, like one-off renders or screen-space calculations. In the latter case it also uses the simplest base class for its views, `FSceneView`.
- `FViewFamilyInfo` for significant rendering work in a real scene.

Its most important fields are:

- `RenderTarget`, the target to render into (presumably `nullptr` if rendering to the screen)
- `Scene`, the scene being viewed
- `Views`, an array of owned views into the scene (see below).

View families can be created and released at will; the operation doesn't look cheap to me but there are many examples of them being constructed as local variables for quick screen-space calculations. Examples of classes which create or own view families:

- A player's HUD actor, `AHUD`
- The game's `UViewportClient` (a singleton except in special editor cases)
- Any `FSceneRenderer` instances (see above).

View

An `FSceneView` is a specific point-of-view for a View Family to render from. I am not sure yet exactly why a view-family would have multiple views, but it's possible this is for offscreen effects which come from derived POV's (like cascaded shadow maps and planar reflections), as well as VR which must display two points of view.

Like view-families themselves, many view instances in a program are just short-term stubs for doing screen-space calculations. If a view is actually used for rendering, then it's usually the child class `FViewInfo`. Renderable views also, as far as I can tell, always need an associated state object `FSceneViewState`. This state is sometimes referred to by its base class `FSceneViewStateInterface`.

Any time you have an `FSceneView` in your rendering pass, it's most likely a full `FViewInfo`. You can unpack it with the following snippet:

```
const FSceneView& view = ...; //Get a view somehow
if (view.bIsViewInfo)
{
    const auto& asViewInfo = static_cast<const FViewInfo&>(view);
    ... //Use the view-info
}
else
{
    //It's some other kind of view type.
    checkf(false, TEXT("Unexpected kind of scene-view!"));
}
```

Custom Per-View Resources

When doing custom render passes, you may need persistent resources for each viewport that uses the pass. There is a class which appears to support this, `ISceneViewFamilyExtentionData` (sic), however it's half-baked and does not work as of 5.3. UE 5.5 has some other interesting things but has not appeared to solve this problem for external users of the engine. Therefore we need our own system for tracking persistent resources per-view.

Fortunately, my library offers a utility for this! `T_EGP_PerViewData`, which will be used in the next article.

GPU Buffers of Scene/View Data

Countless engine shaders reference data about the view and/or scene they are rendering inside. Shaders are explained more in part 4, but I will say here that you can add some common global buffers to your shader's *parameter struct* with this snippet:

```
//Necessary headers:
#include "Runtime/Renderer/Private/SceneRendering.h"

//In the parameter struct (see Part 4 of this series):
    // Tons of camera data, including matrices
    SHADER_PARAMETER_STRUCT_REF(FViewUniformShaderParameters, View)
    // Scene color, scene depth, G-buffer data, and more,
```

```
        //    depending on where in the pipeline your shader executes
        SHADER_PARAMETER_RDG_UNIFORM_BUFFER(FSceneTextureShaderParameters, SceneTex
        // Provides the data in FGPUSceneResourceParameters
        SHADER_PARAMETER_RDG_UNIFORM_BUFFER(FSceneUniformParameters, Scene)
        // Not sure exactly what it provides, but is popular/maybe-required
        //    in MeshMaterial shaders (perhaps LOD data, Instancing, or Nanite).
        // However it can also be null I think?
        SHADER_PARAMETER_STRUCT_INCLUDE(FInstanceCullingDrawParams, InstanceCulling

    //In the setup for your RDG pass:
    const FViewInfo& view = ...; //Got a view somehow
    const FSceneTextureShaderParameters& sceneTextures = ...; //Got the main pass's
    auto* params = graph.AllocParameters<FMyShaderParams>(); //The shader parameter
    params->View = view.ViewUniformBuffer;
    params->Scene = view.GetSceneUniforms().GetBuffer(graph);
    params->SceneTextures = sceneTextures;
    params->InstanceCullingDrawParams = nullptr; //Frankly not sure how to get this
                                                 //  apart from the very specific w
                                                 //  covered in part 7
```

Scene-View Extensions and Custom Passes

While `ENQUEUE_RENDER_COMMAND()` is very useful, it doesn't allow you to change how a scene is rendered. To hook into the render pipeline you need to define and manage your own `FSceneViewExtension`.

Many of the details of this process are covered for you by my library, *ExtendedGraphicsProgramming* or "EGP" for short. See the **Details** section for under-the-hood info. Assuming you'll use my library, define a new class inheriting from `T_EGP_RenderPassSceneViewExtension<P>`. The template parameter should be another custom object you define, inheriting from `U_EGP_RenderPass`, which holds the global parameters for your pass. It's recommended to define these parameters as `UPROPERTY`'s so that your pass can be configured in Blueprints if desired. If any properties are more than POD, they should probably have separate game-thread and render-thread copies.

The SVE base class offers various virtual functions that hook into the Render Thread while it's rendering each viewport. Each callback is for a specific phase of the render pipeline and provides you with an RDG to write commands into. It also provides data specific to each phase; for example `PrePostProcessPass_RenderThread`

will provide you with all the usual inputs to a post-process Material while `PostRenderBasePassDeferred_RenderThread` will provide you with references to the G-buffer for both reading and writing.

Your custom `U_EGP_RenderPass` object will belong to a single World, managed through my plugin's subsystem `U_EGP_RenderPassSubsystem`. You can create or retrieve the pass with

```
auto* myPass = GetWorld()
                    ->GetSubsystem<U_EGP_RenderPassSubsystem>()
                    ->GetPass<UMyRenderPass>(bCreateIfNotExisting)
```

The pass object can override various events, including tick functions in both Game and Render thread. The tick functions will run once per frame, no matter how many views there are.

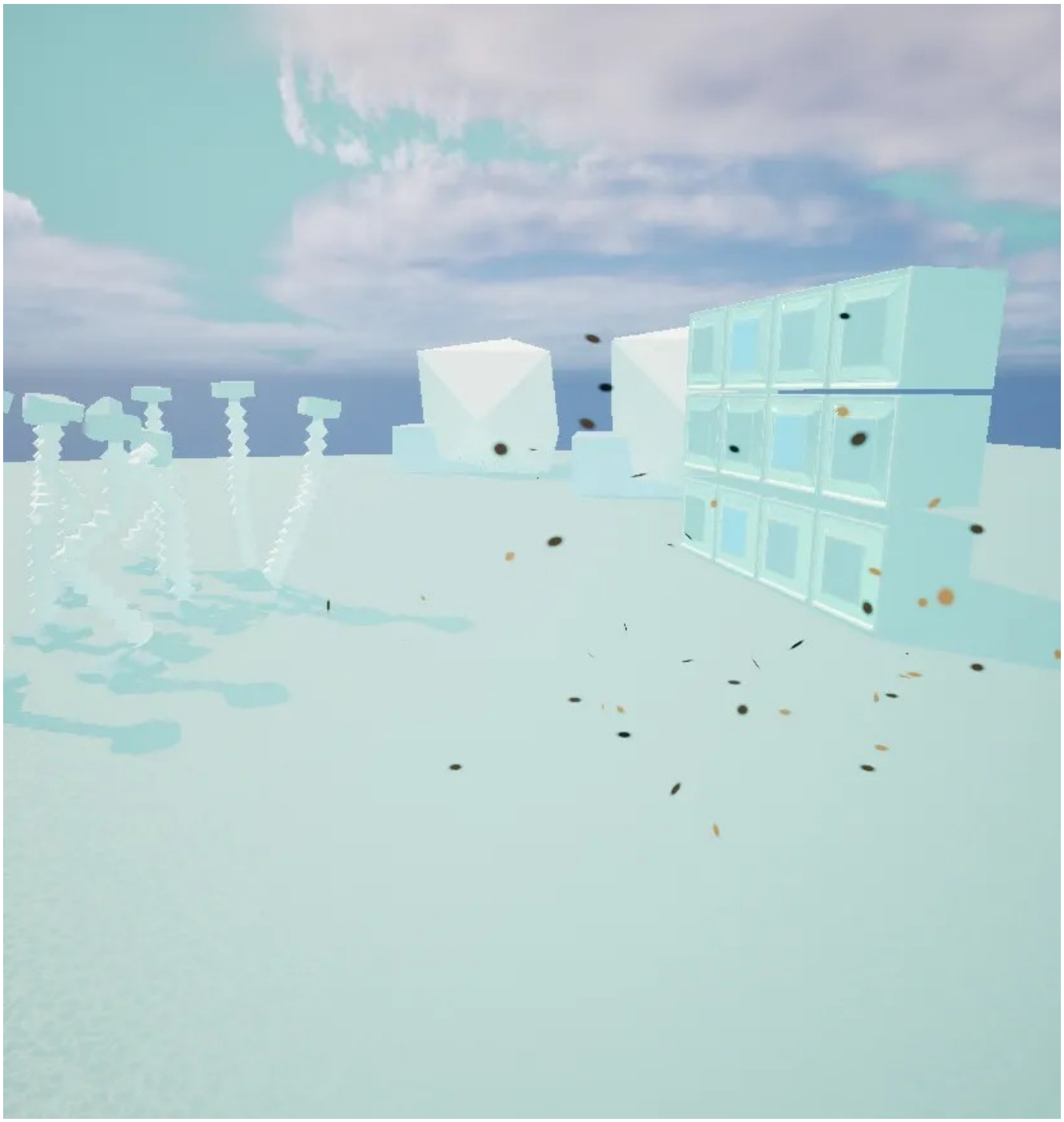
Your pass object is required to at least implement `InitThisPass_GameThread(…)`, to return a constructed instance of your SVE. SVE's must be created by invoking

```
FSceneViewExtensions::NewExtension<FMySVE>(args...)
```

This function will pass all arguments into your SVE's constructor. Refer to the engine for examples of how to define an SVE.

Demo

For this entry we'll set up a scene-view extension using my library and have it randomly clear the albedo g-buffer texture to a solid color, similar to the last demo. The color can optionally be forced through C++/Blueprints, using parameters that are periodically uploaded to the Render Thread.



The demo scene with its albedo cleared to Cyan

First declare your new custom pass in a header:

```
#include "EGP_CustomRenderPasses.h"

UCLASS(BlueprintType)
```

```
class GOL_DEMO_API U_GOL_RenderPass : public U_EGP_RenderPass
{
    GENERATED_BODY()

public:
    UPROPERTY(BlueprintReadWrite, EditAnywhere, meta=(InlineEditConditionToggle
    bool ForceColor = false;
    UPROPERTY(BlueprintReadWrite, EditAnywhere, meta=(EditCondition="ForceColor
    FLinearColor ForcedColor = { 1, 0, 1, 1 };

    TOptional<FLinearColor> GetClearColor_RenderThread() const
    {
       check(IsInRenderingThread());
       return color_renderThread;
    }

protected:
    virtual TSharedRef<F_EGP_RenderPassSceneViewExtension>
                InitThisPass_GameThread(UWorld& thisWorld) override;
    virtual void Tick_GameThread(UWorld& thisWorld, float deltaSeconds) overrid

private:
    TOptional<FLinearColor> color_renderThread;
};
```

Also declare the scene-view extension that executes the pass:

```
struct GOL_DEMO_API F_GOL_PassSVE
    : public T_EGP_RenderPassSceneViewExtension<U_GOL_RenderPass>
{
    //Re-use the parent constructor (which takes a pointer to the pass).
    using T_EGP_RenderPassSceneViewExtension::T_EGP_RenderPassSceneViewExtensio

    //Apply our effect immediately after the deferred pass,
    //    before lighting is computed.
    virtual void PostRenderBasePassDeferred_RenderThread(
       FRDGBuilder& graph, FSceneView& view,
       const FRenderTargetBindingSlots& renderTargets,
       TRDGUniformBufferRef<FSceneTextureUniformParameters> sceneTextures
    ) override;
};
```

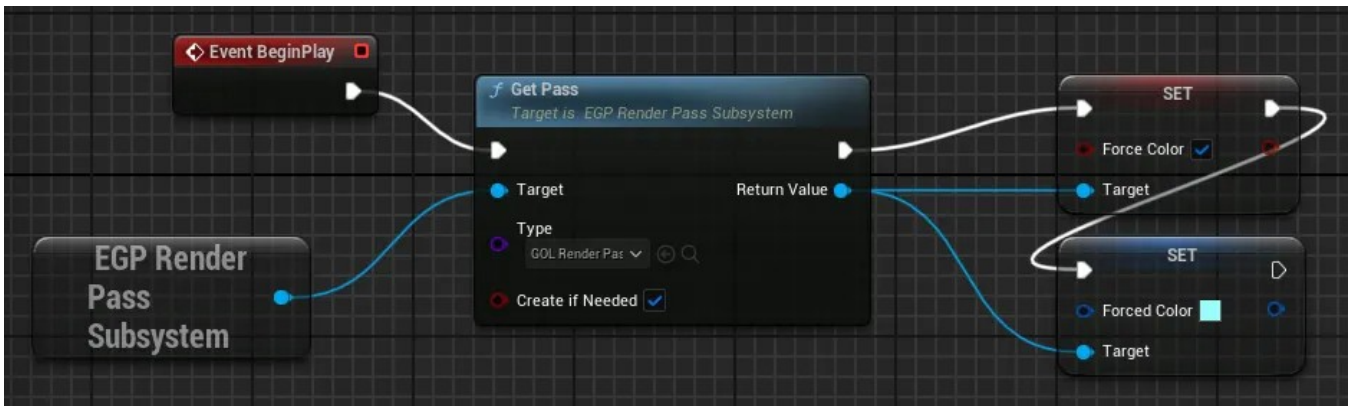
Finally, define both of these objects in a cpp file.

```
#include "GOL_RenderPass.h"
#include "RenderGraphUtils.h"

TSharedRef<F_EGP_RenderPassSceneViewExtension> U_GOL_RenderPass::
    InitThisPass_GameThread(UWorld& thisWorld)
{
    //By default, only apply this pass to the first player-controller's view.
    ViewFilter->FilterByPlayerIdx(0);

    return FSceneViewExtensions::NewExtension<F_GOL_PassSVE>(this);
}
void U_GOL_RenderPass::Tick_GameThread(UWorld& thisWorld, float deltaSeconds)
{
    //Pick the new clear color (or null to not clear this frame).
    TOptional<FLinearColor> newClearColor;
    if (ForceColor || FMath::FRand() < 0.09f)
    {
       newClearColor = (ForceColor ?
                        ForcedColor :
                        FLinearColor{ FMath::FRand(), FMath::FRand(), FMath::FRan
    }

    //Copy the color and field pointer into a lambda and send that lambda to th
    //This prevents race conditions from corrupting that color value.
    auto* outputPtr = &color_renderThread;
    auto updateClearColor = [outputPtr, newClearColor](FRHICommandListImmediate
       *outputPtr = newClearColor;
    };
    ENQUEUE_RENDER_COMMAND(UpdateClearColor)(MoveTemp(updateClearColor));
}

void F_GOL_PassSVE::PostRenderBasePassDeferred_RenderThread(
    FRDGBuilder& graph, FSceneView& view,
    const FRenderTargetBindingSlots& renderTargets,
    TRDGUniformBufferRef<FSceneTextureUniformParameters> sceneTextures
) {
    auto tryClearColor = Pass->GetClearColor_RenderThread();
    auto sceneColor = renderTargets.Output[0].GetTexture();

    if (tryClearColor.IsSet() && sceneColor != nullptr)
        AddClearRenderTargetPass(graph, sceneColor, *tryClearColor);
}
```

Lastly we need to ensure the pass is created by calling (either in C++ or Blueprints):

```
U_EGP_RenderPassSubsystem::GetPass(U_GOL_RenderPass::StaticClass(), true);
```

I did it in the demo level's Level Blueprint, and also forced it to always use Cyan:



Create the pass on level start, and force it to use Cyan

When you press Play you should see the opaque parts of the screen flicker in Cyan. Below is what it looks like with 'Force Color' turned off.

WARNING: the flickering is slowed down here; it's faster in the demo

Code

The SVE is added to the demo repo in the *part/3* branch:

heyx3/UnrealExtendedGraphicsProgrammingDemo at part/3

Details

Per-View Resource Management

My library manages custom per-view resources by performing three things:

- Lazy-initialization of the data each time a new view is found
- Resampling of the data whenever the screen resolution changes
- Automatic cleanup of the data after a certain number of frames, because I can't for the life of me find any way to know when a view has officially been destroyed.

Views and view-families are recreated each frame, so you can't index the data by pointer! Instead use their persistent ID:

```
auto viewID = view.State->GetViewKey();
```

Scene/View workflows

Many of the basic scene/view functions are in the renderer module object itself (gettable with `GetRendererModule()`). For example check out `CreateAndInitSingleView(…)` or `AllocateViewState(…)`. You likely won't ever need to call these functions but reading them can help you understand how the various responsibilities of the rendering module are organized.

It's not stated explicitly anywhere, but based on engine code I believe you are supposed to create new SVE's on the Game Thread only. If your subsystem is the sole owner of an SVE through a `TSharedPtr<>`, you can prematurely kill the SVE with `myPtr.Reset()`.

Implementing a 3D scene pass in an SVE is complicated, even with the help of my library, and won't be covered until the 7th and final article in this series. You first need to be comfortable with how Unreal does shaders.

Your SVE can override functions to hook into various phases in the rendering of any and all viewports. **In the editor this includes thumbnails and Material previews**, so be careful! If your render pass does extreme things then it's recommended to add custom predicates to the array `IsActiveThisFrameFunctions`, to be selective about where it executes. You can examine the current view for its player-index, camera Actor, or Render-Target, and accordingly decide whether to execute on that view.

Setting up an SVE without my library

Your SVE's constructor should take a `const FAutoRegister&`, an empty type that only exists as a label, followed by any custom arguments you want to pass to it on initialization. Then you must create it by calling

```
//On the game thread:
TSharedPtr<FMySVE, ESPMode::ThreadSafe> mySVE =
```

```
FSceneViewExtensions::NewExtension<FMySVE>(constructorArg2, constructorArg3
```

The easiest way to manage its lifetime is through a World Subsystem (Unreal's solution for per-world singletons), creating your extension at the start of the world and owning the shared pointer so that the SVE automatically unregisters and destructs when the subsystem is destroyed.

With this setup your subsystem should manage all the data related to rendering, including game-thread `UPROPERTY`'s that upload themselves to the render-thread on Tick, while the SVE simply pulls that render-thread data as needed.

Warning: *If your SVE is destroyed when your subsystem is destroyed, be wary of race conditions from the SVE accessing subsystem parameters! My library adds a GPU fence to subsystem destruction which hangs until all pass objects and SVE's have cleaned themselves up.*

SVE's actually live at global/engine scope, so if you go with the above method you also need to add new conditions to the inherited field `IsActiveThisFrameFunctions` to only be active within its owning World. You can add any other conditions you want to that field, such as only enabling at run-time or never enabling for little thumbnail preview editor worlds.

Unreal Engine 5 | Graphics Programming | Shaders | Technical Art

Edit profile

Written by William Mishra-Manning

59 followers · 5 following

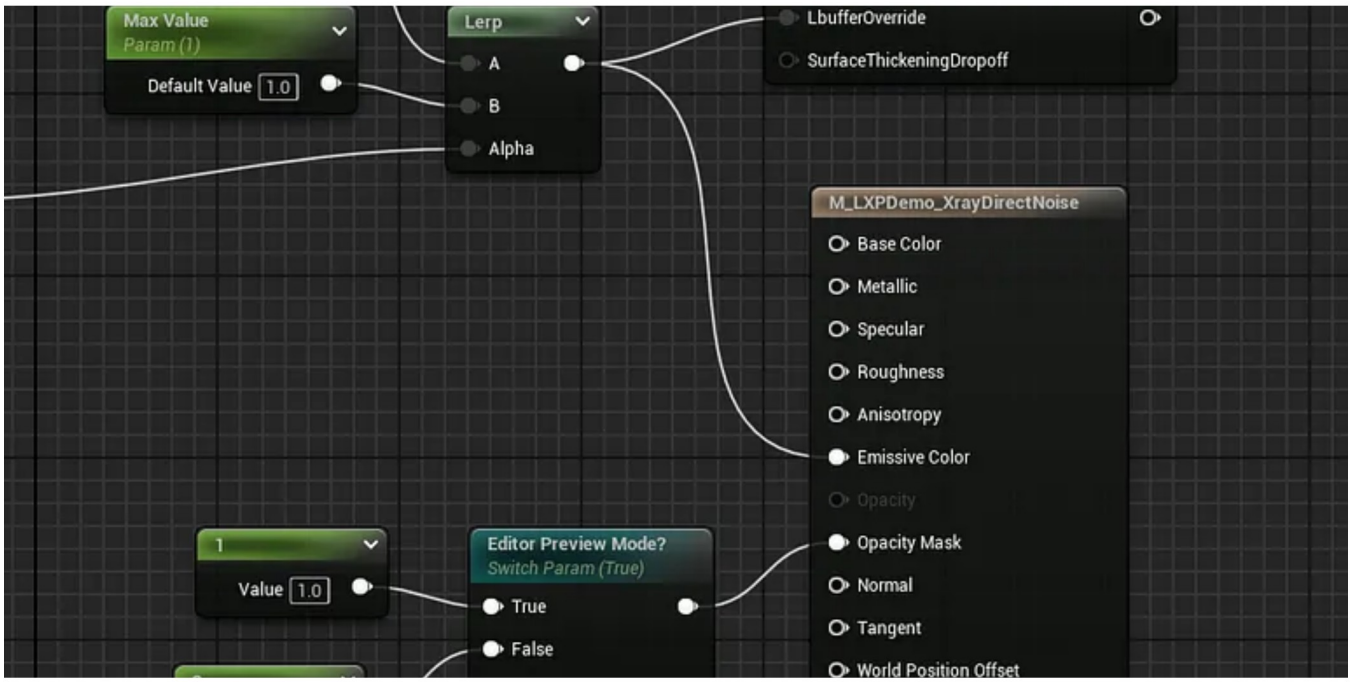
Graphics Programming, Procedural Generation

No responses yet

William Mishra-Manning

What are your thoughts?

More from William Mishra-Manning

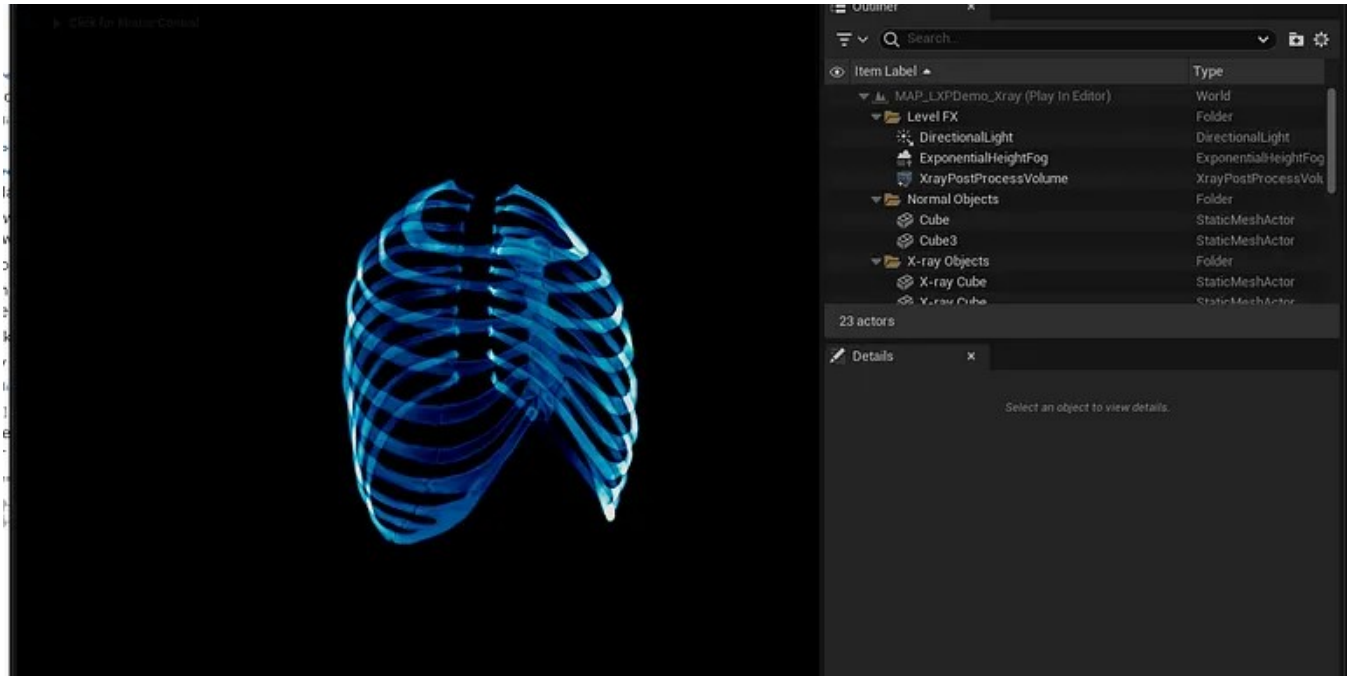


William Mishra-Manning

Advanced Graphics Programming in Unreal, part 6

This is a seven-part article series.

Jun 25, 2025 1 2

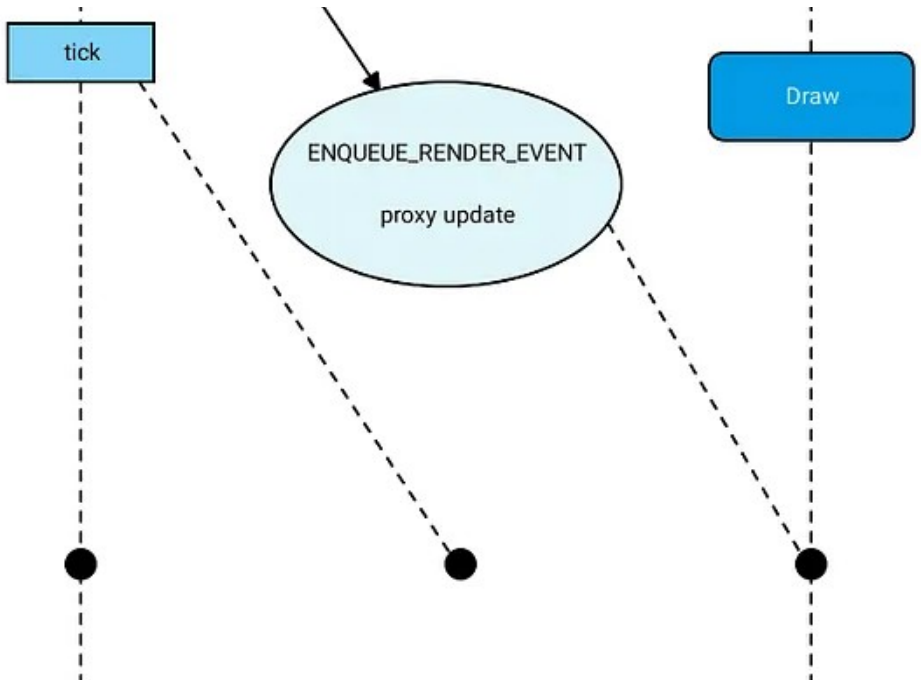


William Mishra-Manning

Advanced Graphics Programming in Unreal, part 1

This is a seven-part article series.

Jun 24, 2025 31 1

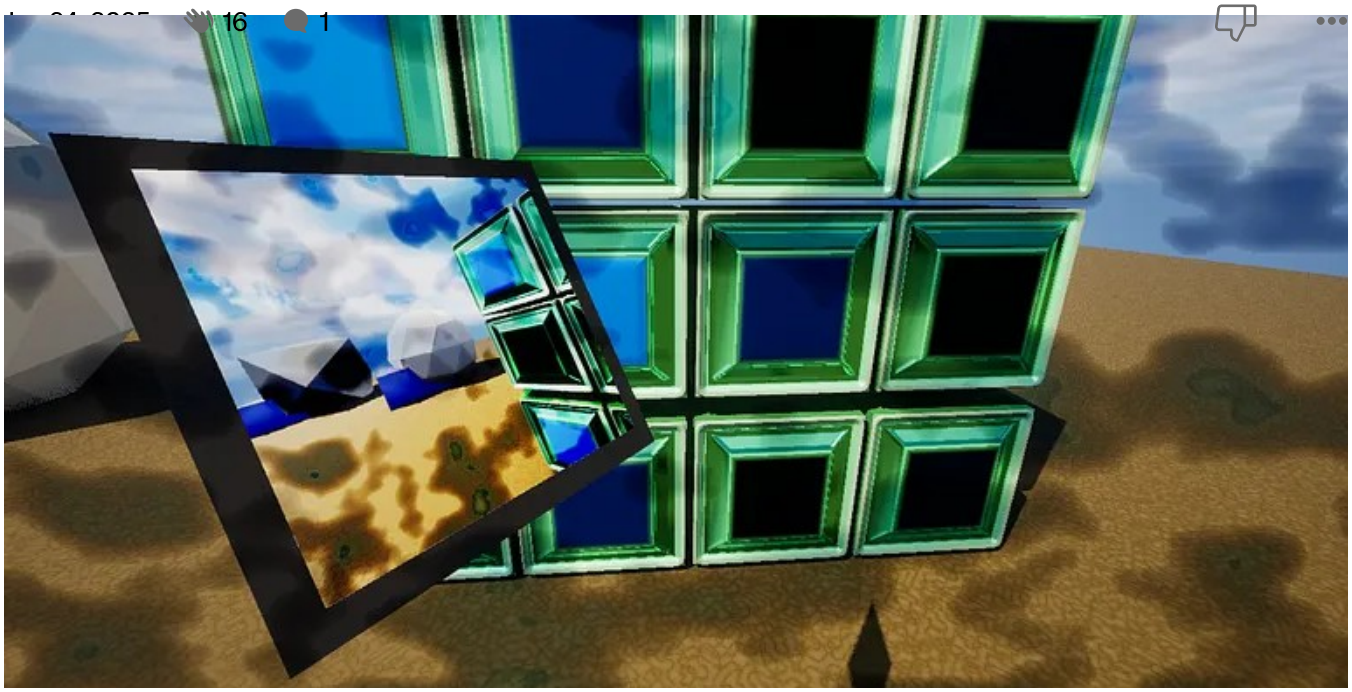


William Mishra-Manning

Advanced Graphics Programming in Unreal, part 2

This is a seven-part article series.

16 1



William Mishra-Manning

Advanced Graphics Programming in Unreal, part 4

This is a seven-part article series.

Jun 25, 2025 18

See all from William Mishra-Manning

Recommended from Medium

AmirHossein Aghajari

Liquid Glass: iOS Effect Explanation

Apple's Liquid Glass effect with shaders: math, distortion & chromatic aberration

Nov 23, 2025 240 1

Nikhil Maurya

RDG in Practice: From Shader Parameters to a Working Pass

The Render Dependency Graph (RDG) is Unreal Engine's way of describing GPU work : passes, resources and lifetimes. So the renderer can...

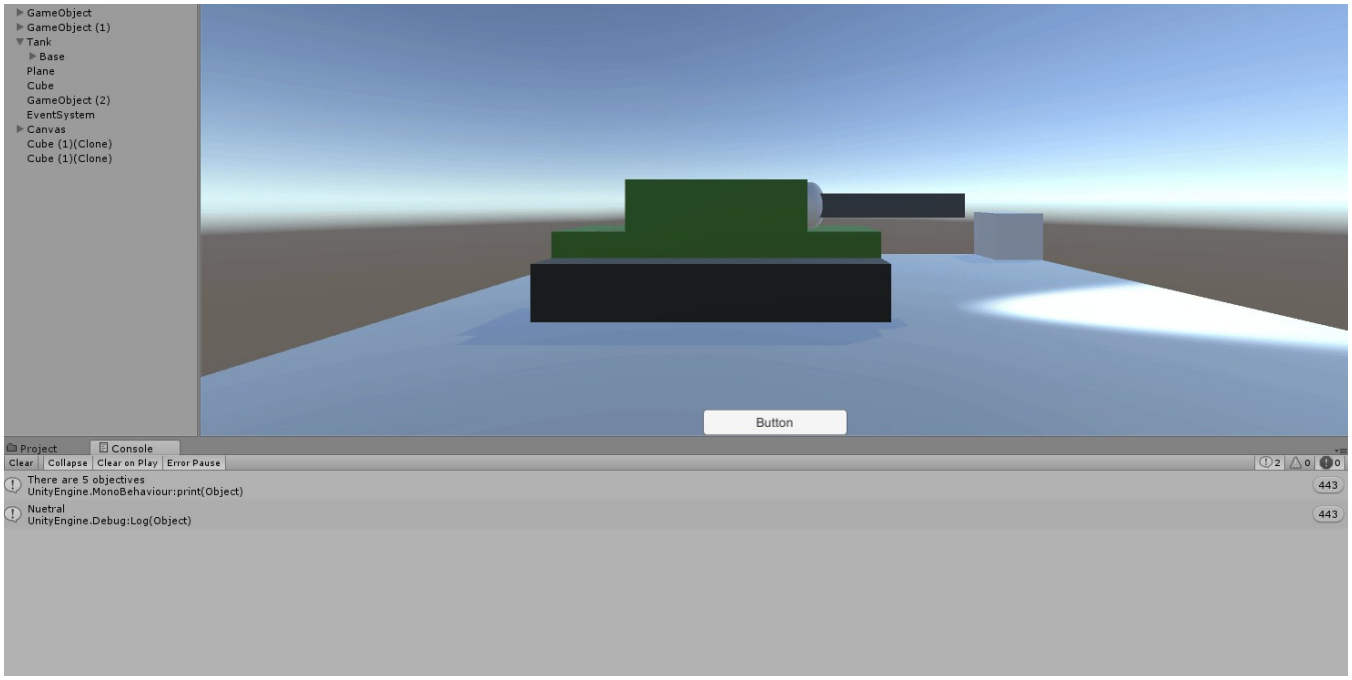
4

Akash Kadam

Setting Up FreeRTOS on STM32 / ESP32 from Zero A Hands-On Beginner's Guide

If you've already understood what FreeRTOS is and why bare-metal eventually breaks, the next natural question is:

Jan 29 1



Zac Bogner

Switching from Unity to Unreal

In the previous article, I wrote about Switch Statements in Unity.

Dec 22, 2025 3

1. **Start with Bayes' Rule:**

$$P(x|z) = \frac{P(z|x)P(x)}{P(z)}$$

2. **Divide by the "Free" state ($\neg x$) to create an Odds Ratio:**

$$\frac{P(x|z)}{P(\neg x|z)} = \frac{P(z|x)}{P(z|\neg x)} \cdot \frac{P(x)}{P(\neg x)}$$

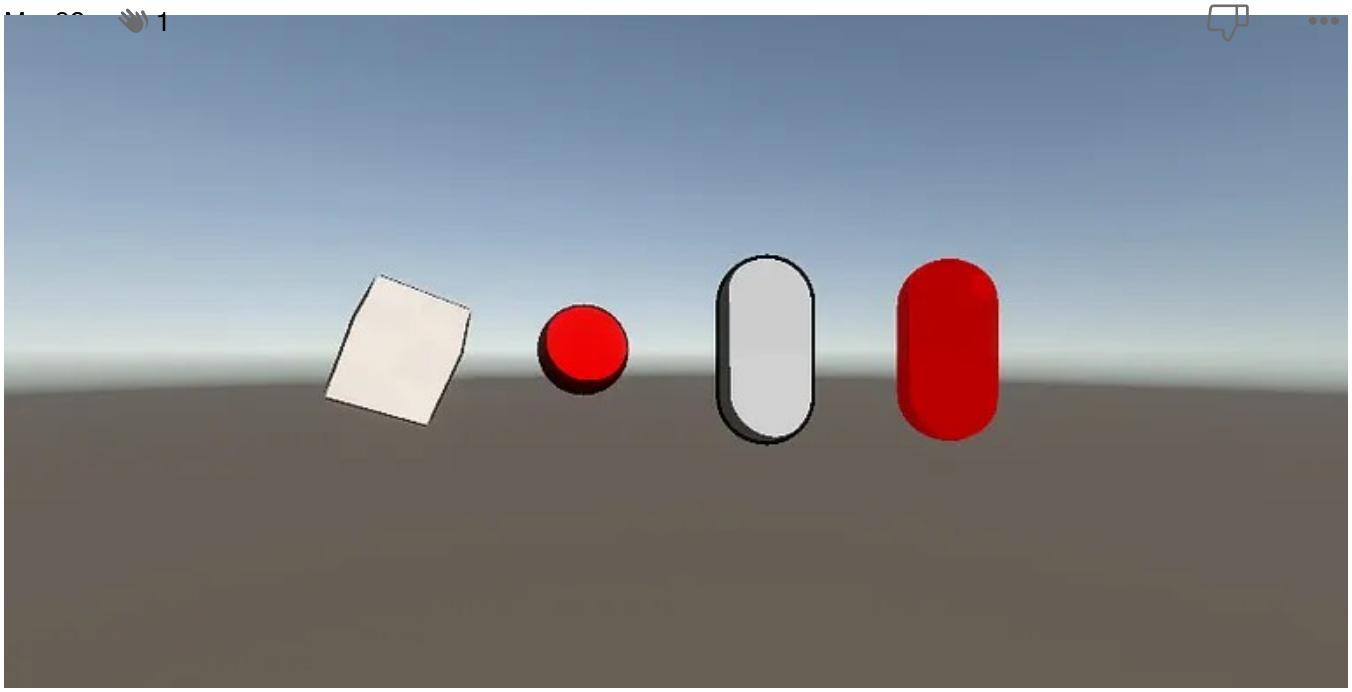
3. **The Result:** This allows the robot to update its map by comparing how likely a measurement is if a wall exists ($P(z|x)$) versus if it doesn't ($P(z|\neg x)$).

Naren Suri

Occupancy Grid Mapping Algorithm

Occupancy Grid Mapping serves as a foundational framework in robotic perception, enabling a mobile agent to represent an unstructured...

1



madcritter

Toon Shaders in Unity — From Shader Graph to Custom HLSL

Toon shading — also known as cel shading — is one of the most iconic visual styles in gaming and animation. From anime-inspired games to...

Nov 23, 2025 1

See more recommendations